

1주 차 보고서

(2026년 4 월 5 일 ~ 2026년 4 월 11 일)

작성자

박신우

이번 주 회의 내용

- 중간발표까지 목표 정리
- 서버 캐릭터 이동 확인
- 아이템,캐릭터 스텟 정리

이번 주 한 일

4.7 언리얼 시네마틱 공부

1스테이지에서 2스테이지로 넘어갈 때 시네마틱 같은 것이 있으면 좋겠다는 생각이 들어서, 시네마틱에 관련된 지식이 없어, 자료를 찾아보며 공부를 하는 시간은 가졌다.

4.8 회의 진행

회의 때 시네마틱은 중간발표때까지 완성하기 힘들 것 같다는 결론이 나와서, 미루어 두기로 하였다.

4.9~10 트럭으로 좀비 치기.

좀비를 트럭으로 쳤을 때 좀비가 날아가게 하는 함수를 만들어 주었다.

OnTruckMeshHit() 함수에서 좀비가 날아가게 하는 함수를 호출하게 만들었다.

```
void ATruck::OnTruckMeshHit(UPrimitiveComponent* HitComponent, AActor* OtherActor, UPrimitiveComponent* OtherComp, FVector NormalImpulse, const FHitResult& Hit)
{
    // 유효한 액터인지 검사
    if (!OtherActor || OtherActor == this)
    {
        return;
    }
    // 좀비가 아니거나 죽었으면 무시
    ABaseZombie* Zombie = Cast<ABaseZombie>(OtherActor);
    if (!Zombie || !Zombie->IsAlive())
    {
        return;
    }
    ProcessZombieImpact(Zombie, Hit.ImpactPoint, GetVelocity().GetSafeNormal(), GetVelocity().Size());
}
```

OnTruckMeshHit()함수는 시작할 때 메시에 OnComponentHit를 바인딩해서 실제 물리 충돌을 받도록 하였다.

```
void ATruck::BeginPlay()
{
    Super::BeginPlay();

    if (USkeletalMeshComponent* TruckMesh = GetMesh())
    {
        TruckMesh->SetNotifyRigidBodyCollision(true);
        TruckMesh->OnComponentHit.AddDynamic(this, &ATruck::OnTruckMeshHit);
    }
}
```

우선적으로, 매 프레임마다 검사를 해버리면 좀비를 여러 번 치면서 버그가 일어나기 때문에, 좀비의 충돌시간을 업데이트 하여 좀비를 여러 번 치는 것을 방지해 주었다.

```
void ATruck::ProcessZombieImpact(ABaseZombie* Zombie, const FVector& ImpactPoint, const FVector& ImpactDirection, float ImpactSpeed)
{
    // 속도가 느리거나 좀비가 죽으면 무시
    if (!Zombie || !Zombie->IsAlive() || ImpactSpeed < ZombieImpactMinSpeed)
    {
        return;
    }

    // 좀비를 여러 번 쳐버리는 것을 방지
    const float CurrentTime = GetWorld()->GetTimeSeconds();
    if (const float* LastImpactTime = LastZombieImpactTimes.Find(Zombie))
    {
        if (CurrentTime - *LastImpactTime < ZombieImpactCooldown)
        {
            return;
        }
    }

    // LastZombieImpactTimes 맵에 좀비의 마지막 충돌 시간을 업데이트
    LastZombieImpactTimes.Add(Zombie, CurrentTime);
}
```

이번 주 한 일

그 후 방향 계산에 실패해도 좀비를 앞으로 날릴 수 있도록 방향을 보정해준 후, 좀비가 입을 데미지와 좀비를 날아가게 하는 넉백을 속도에 따라 계산해주었다.

```
// 전달받은 충돌 방향이 거의 0 이면 트럭 전방으로 간주
const FVector SafeImpactDirection = ImpactDirection.IsNearlyZero() ? GetActorForwardVector() : ImpactDirection.GetSafeNormal();

// 좀비가 받을 데미지 속도에 따라 계산.
const float Damage = FMath::GetMappedRangeValueClamped(
    FVector2D(ZombieImpactMinSpeed, ZombieImpactFatalSpeed),
    FVector2D(ZombieImpactMinDamage, ZombieImpactMaxDamage),
    ImpactSpeed);

// 좀비의 넉백을 속도에 따라 조절
const float KnockbackScale = FMath::GetMappedRangeValueClamped(
    FVector2D(ZombieImpactMinSpeed, ZombieImpactFatalSpeed),
    FVector2D(1.0f, 1.6f),
    ImpactSpeed);
```

이 함수에서 사용한 `GetMappedRangeValueClamped` 함수는 `InputRange` 범위에 대하여, `OutputRange` 백분율로 `value`를 리턴해주는 함수이다.

```
template<typename T, typename T2>
[[nodiscard]] static FORCEINLINE auto GetMappedRangeValueClamped(const UE::Math::TVector2<T>& InputRange, const UE::Math::TVector2<T>& OutputRange, const T2 Value)
{
    using RangePctType = decltype(T() * T2());
    const RangePctType ClampedPct = Clamp<RangePctType>(GetRangePct(InputRange, Value), 0.f, 1.f);
    return GetRangeValue(OutputRange, ClampedPct);
}
```

그 후 좀비의 위치를 가져와 트럭의 로컬 좌표로 변경을 해주고, 좀비가 트럭의 몸체에 부딪혔는지 검사를 해준다.

애초에 `OnTruckMeshHit()`에서 이 함수를 호출 해주는데, 검사를 해주는 이유는 `OnTruckMeshHit()`는

1. 좀비가 트럭에 스친 거 일 수 있어서.
2. 후에 나올 즉사조건을 더 자연스럽게 만들기위해서.

이렇게 2가지 이유 때문이다.

```
// 좀비의 위치를 트럭의 로컬 좌표로 변경
const FVector LocalZombieLocation = GetActorTransform().InverseTransformPosition(Zombie->GetActorLocation());
// 트럭 메쉬 크기를 가져옴
const FVector MeshExtent = GetMesh() ? GetMesh()->Bounds.BoxExtent : FVector(150.0f, 100.0f, 100.0f);
// 좀비가 트럭 몸체 범위 안에 있는지 검사
const bool bWithinTruckLength =
    LocalZombieLocation.X > -MeshExtent.X * 1.15f &&
    LocalZombieLocation.X < MeshExtent.X * 1.15f;
const bool bWithinTruckWidth = FMath::Abs(LocalZombieLocation.Y) < MeshExtent.Y * 1.35f;
const bool bWithinTruckHeight = FMath::Abs(LocalZombieLocation.Z) < MeshExtent.Z * 1.5f;
const bool bTruckBodyImpact = bWithinTruckLength && bWithinTruckWidth && bWithinTruckHeight;
```

그 후 좀비에게 데미지를 주고, 특정 속도 이상이면 즉사를 하도록 조건을 만들었다.

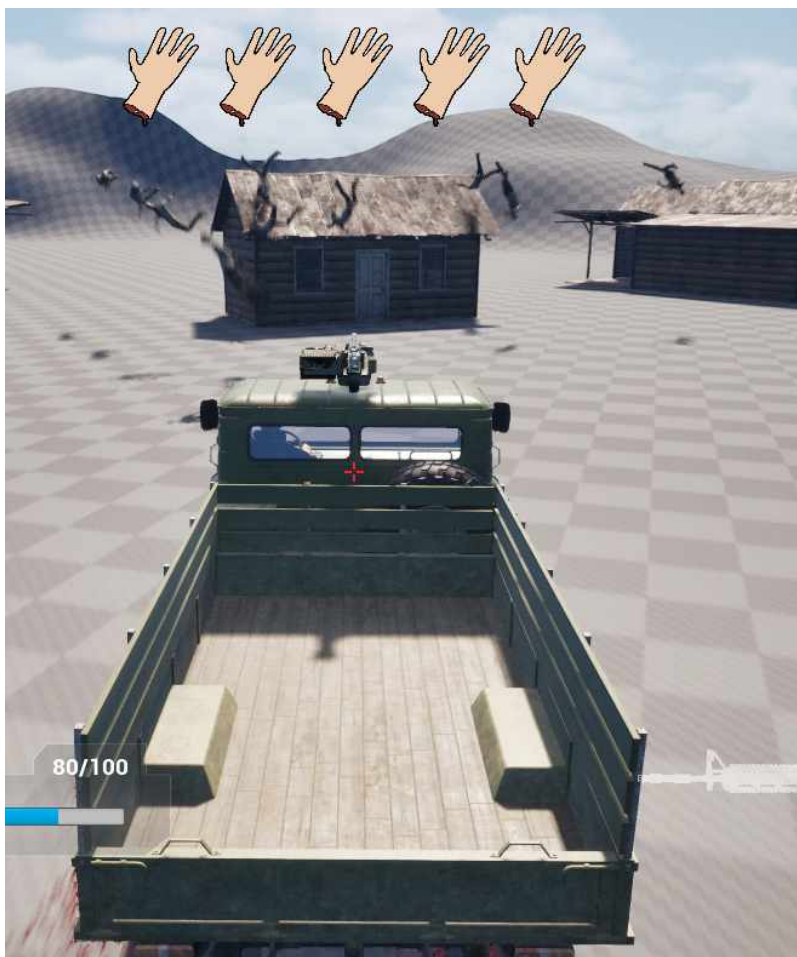
```
// 좀비에게 데미지를 줌
UGameplayStatics::ApplyPointDamage(
    Zombie,
    Damage,
    ImpactFlingDirection,
    DamageHit,
    GetController(),
    this,
    nullptr);
// 좀비 즉사 조건
const bool bCheatFlingImpact = (bTruckBodyImpact) && ImpactSpeed >= ZombieImpactMinSpeed;
```

이번 주 한 일

이제 죽사 조건이 성립되었다면 바로 죽이고,
살아있다면 캐릭터를 LaunchCharacter를 활용해 넉백 시키고,
죽은 경우에 레그돌을 하여 임펄스를 가해 좀비를 날려 버렸다.

```
if (Zombie->IsAlive() &&  
    (ImpactSpeed >= ZombieImpactFatalSpeed ||  
     bCheatFlingImpact))  
{  
    Zombie->Die();  
}  
  
// 살아있다면 캐릭터를 넉백  
if (Zombie->IsAlive())  
{  
    const FVector LaunchVelocity =  
        ImpactFlingDirection * (ZombieImpactKnockback * KnockbackScale * 1.2f) +  
        FVector::UpVector * (ZombieImpactUpwardKnockback * KnockbackScale);  
    Zombie->LaunchCharacter(LaunchVelocity, true, true);  
    return;  
}  
  
// 죽은 경우 레그돌 하여 넉백  
if (USkeletalMeshComponent* ZombieMesh = Zombie->GetMesh())  
{  
    const FVector WorldImpulse =  
        ImpactFlingDirection * (ZombieImpactImpulse * KnockbackScale * 1.2f) +  
        FVector::UpVector * (ZombieImpactImpulse * 0.2f * KnockbackScale);  
    ZombieMesh->AddImpulseAtLocation(WorldImpulse, ImpactPoint, FName(TEXT("pelvis")));  
}
```

시원하게 날리는 모습을 확인 할 수 있었다.



다음 주 할 일

- 좀비 행동트리
- 좀비 애니메이션 동기화

특이사항

시험공부 때문에 많이 진행하지 못하였다.

15주 차 보고서

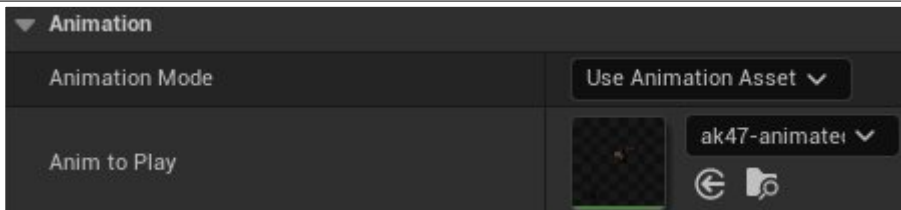
(2026년 4 월 5 일 ~ 2026년 4 월 11 일)

작성자

배주환

이번 주 회의 내용

이번 주 한 일



총 애니메이션을 적용하려고 보니까 테스트용으로 애니메이션이 한번만 적용이 되도록 해놓고 그 총을 맵에다가 두는 방식을 사용했다. 신우와 전화를 했더니 총에 애니메이션이 적용되어도 에셋을 잘못사서 밑에 이상한게 보인다고 해서 다른 총으로 바꾸기로 했다.


```

bool Room::HandleEnterPlayer(PlayerRef player)
{
    bool success = EnterRoom(player, true);

    if (success == false)
        return false;

    Protocol::S_SPAWN_ITEM spawnItemPkt;
    Protocol::ObjectInfo* itemInfo = spawnItemPkt.add_items();

    itemInfo->set_object_id(1); // 이 무기의 고유 번호는 1번!
    itemInfo->set_object_type(Protocol::OBJECT_TYPE_ITEM);
    itemInfo->set_weapon_type(Protocol::WEAPON_TYPE_RIFLE);

    Protocol::PosInfo* pos = itemInfo->mutable_pos_info();
    pos->set_x(-860.0f);
    pos->set_y(-180.0f);
    pos->set_z(30.0f);
    pos->set_yaw(0.0f);

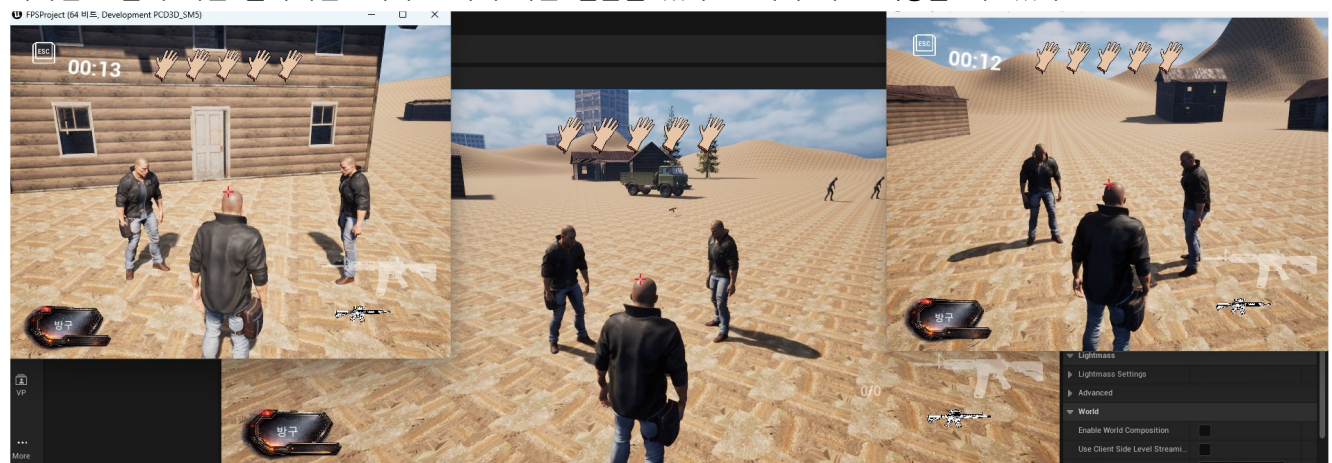
    // 방금 접속한 플레이어에게 패킷 전송
    SendBufferRef itemBuffer = ServerPacketHandler::MakeSendBuffer(spawnItemPkt);
    player->session.lock()->Send(itemBuffer);

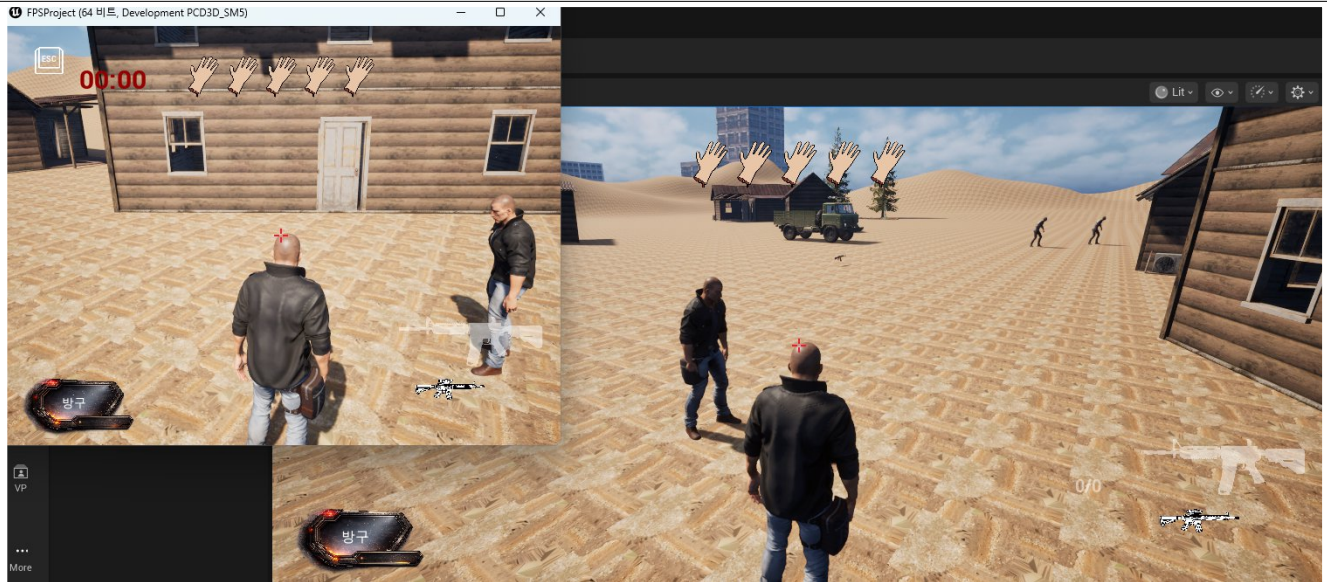
    return EnterRoom(player, true);
}

```

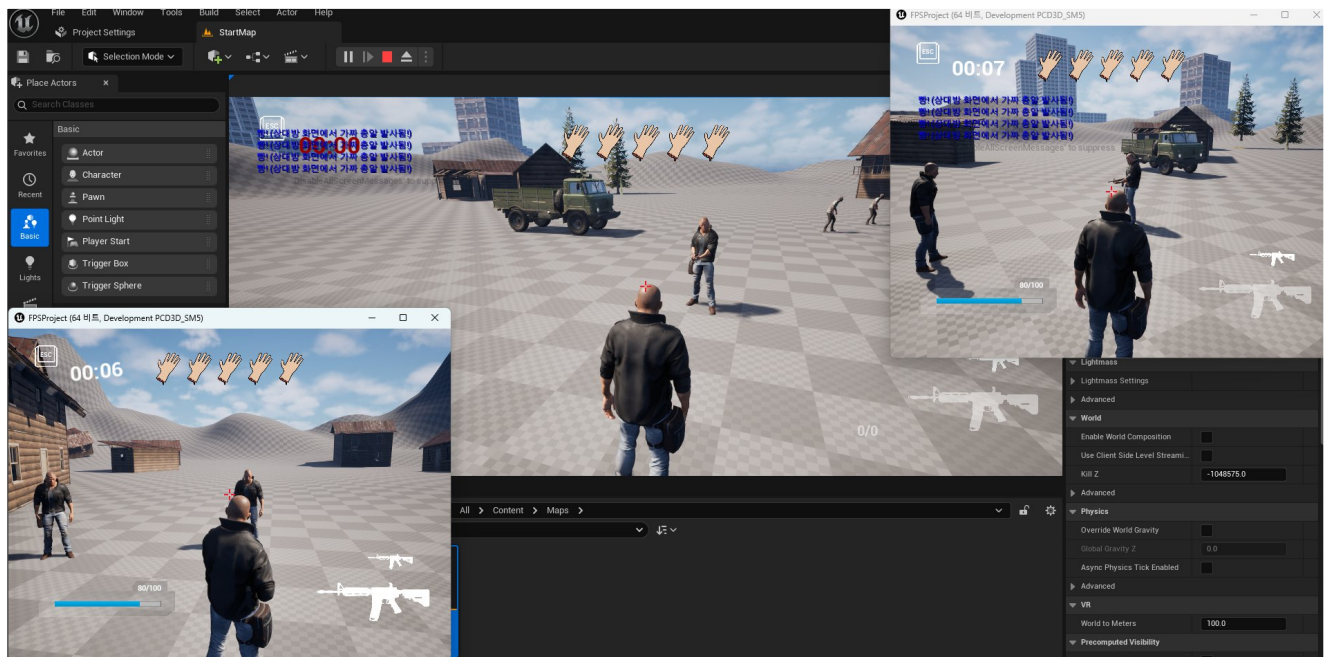
지금은 총을 맵에다가 두는 방식이 아니라 플레이어가 방에 들어가면 서버에서 지정한 좌표로 뿌려주고 그 위치에 아이템이 생성되는 방식으로 수정을 했다. 현재 맵에는 아이템이 총 하나인 상태라 이렇게 코드를 짰는데 아이템이 여러 개가 되면 서버에서 아이템을 관리하는 시스템을 만들어야 할 것 같다.

게임서버프로그래밍 시간에 체스게임을 만드는데 클라이언트가 접속을 끊었을 때 해당 플레이어가 사라지게 하고, 사라진 모습이 다른 클라이언트에서 보이게 하는 실습을 했다. 그래서 바로 적용을 해보았다.





클라 하나를 종료하면 다른 화면에서도 적용이 되는 모습이다.



총 에셋을 바꾸고 총을 주웠을 때 보이지 않는 문제가 있었는데 전에 있던 총의 크기를 0.15배 해놓아서 안보였던 거였다. 다시 1배로 하니깐 보였다. 총 발사한것도 브로드캐스트하여서 총을 쏜 캐릭터 화면 외에 나머지 화면에 땡! 하는 로그가 보이는 모습이다.

이제 총 동기화는 끝났는데 전체적인 흐름은 이렇다.

1. 서버에서 플레이어가 방에 접속하면 맵에 배치될 아이템 정보를 담은 패킷 전송, 클라는 지정된 좌표에 무기 스폰 및 관리를 위해 FieldItems맵에 아이템 ID와 액터 포인터 등록
2. 클라에서 플레이어가 무기를 주우면 서버로 패킷 전송, 서버는 해당 플레이어의 무기 정보를 갱신하고, 방 안의 모든 유저에게 브로드캐스트
3. 패킷을 받은 클라는 기존 아이템 삭제, 캐릭터 손에 새로운 무기 스폰 및 소켓에 부착
4. 클라에서 사격 시 서버로 패킷 전송, 서버는 패킷을 브로드캐스트
5. 다른 클라 화면에는 RemoteFire함수가 호출


```

message C_ENTER_TRUCK
{
    uint64 truck_id = 1;
    TruckSeatType seat_type = 2;
}

message S_ENTER_TRUCK
{
    uint64 player_id = 1;
    uint64 truck_id = 2;
    TruckSeatType seat_type = 3;
}

message C_EXIT_TRUCK
{
}

message S_EXIT_TRUCK
{
    uint64 player_id = 1;
    uint64 truck_id = 2;
    TruckSeatType seat_type = 3;
}

message C_TRUCK_MOVE
{
    PosInfo info = 1;
}

message S_TRUCK_MOVE
{
    PosInfo info = 1;
}

```

```

enum TruckSeatType
{
    TRUCK_SEAT_NONE = 0;
    TRUCK_SEAT_DRIVER = 1;
    TRUCK_SEAT_CARGO = 2;
    TRUCK_SEAT_TURRET = 3;
}

```

트럭에 관한 프로토콜이다. 운전석, 트렁크, 총 쏘는 자리로 자리를 나누어서 만들었고, 트럭에 탔을 때, 내렸을 때, 트럭을 운전할 때의 패킷을 만들었다.

Handle_C_ENTER_TRUCK, Handle_C_EXIT_TRUCK, Handle_C_TRUCK_MOVE 패킷이 있는데 패킷 핸들러의 동작원리는 이렇다.

```

auto gameSession = static_pointer_cast<GameSession>(session);

```

PacketSessionRef를 GameSession으로 변환한다.

```

PlayerRef player = gameSession->player.load();
if (player == nullptr)
    return false;

```

세션과 연결된 플레이어 객체가 있는지 스레드 세이프하게 확인한다. 로그인 처리가 안 끝났거나 이미 로그아웃 중인 비정상적인 요청을 걸러낸다.

```

RoomRef room = player->room.load().lock();
if (room == nullptr)
    return false;

```

플레이어가 어떤 방에 들어가 있는 지 확인한다. 룸의 참조를 weak_ptr로 관리하여서 플레이어와 룸이 서로를 shared_ptr로 물고 있는 것을 방지한다.

```
room->DoAsync(&Room::HandleTruckMove, player, Protocol::C_TRUCK_MOVE(pkt));
```

룸의 일감 큐에 DoAsync를 통해 트럭에 관련된 처리를 부탁하고 빠지면 룸을 관리하는 스레드가 순서대로 일을 처리한다.

Room 클래스에서는 HandleEnterTruck, HandleExitTruck, HandleTruckMove 함수를 만들었다.

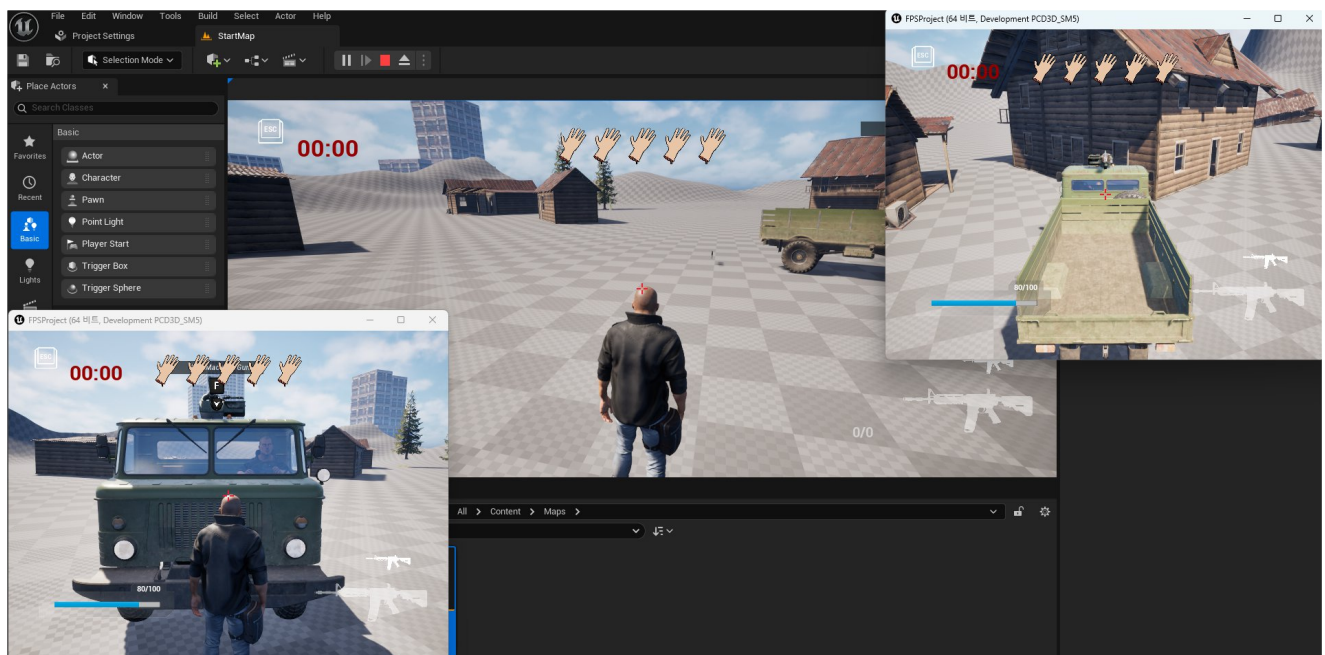
HandleEnterTruck : 클라가 탑승 요청 시 차량의 점유 상태를 검증하고, 이미 다른 플레이어가 있으면 이를 차단한다. 검증이 끝나면 트럭 상태 객체와 플레이어 객체에 서로의 정보를 기록하여 탑승 상태로 만든다.

HandleExitTruck : 플레이어가 내릴 때 트럭과 플레이어쪽의 탑승 상태를 지운다.

HandleTruckMove : 운전자만 트럭을 움직일 수 있게 하고, 트럭의 위치를 플레이어의 위치로 만든다.

클라에서는 S_ENTER_TRUCK, S_EXIT_TRUCK, S_TRUCK_MOVE에서 엔진의 게임 스레드로 작업을 넘겨준다. 함수에서 수신된 패킷 데이터를 new로 복사하고, AsyncTask를 사용하여 처리를 메인 게임 스레드로 넘긴다. 네트워크 스레드에서 액터의 위치를 옮겼을 때 크래시가 나는 것을 방지하기 위함이다.

게임 스레드 내부에서 현재 월드의 gameinstance를 찾아서 실제 처리 함수를 호출하고 복사했던 패킷 메모리를 해제한다.



다음 주 할 일

문 열릴 때 상대방 화면에 브로드캐스트, 트럭에 달린 기관총 브로드캐스트

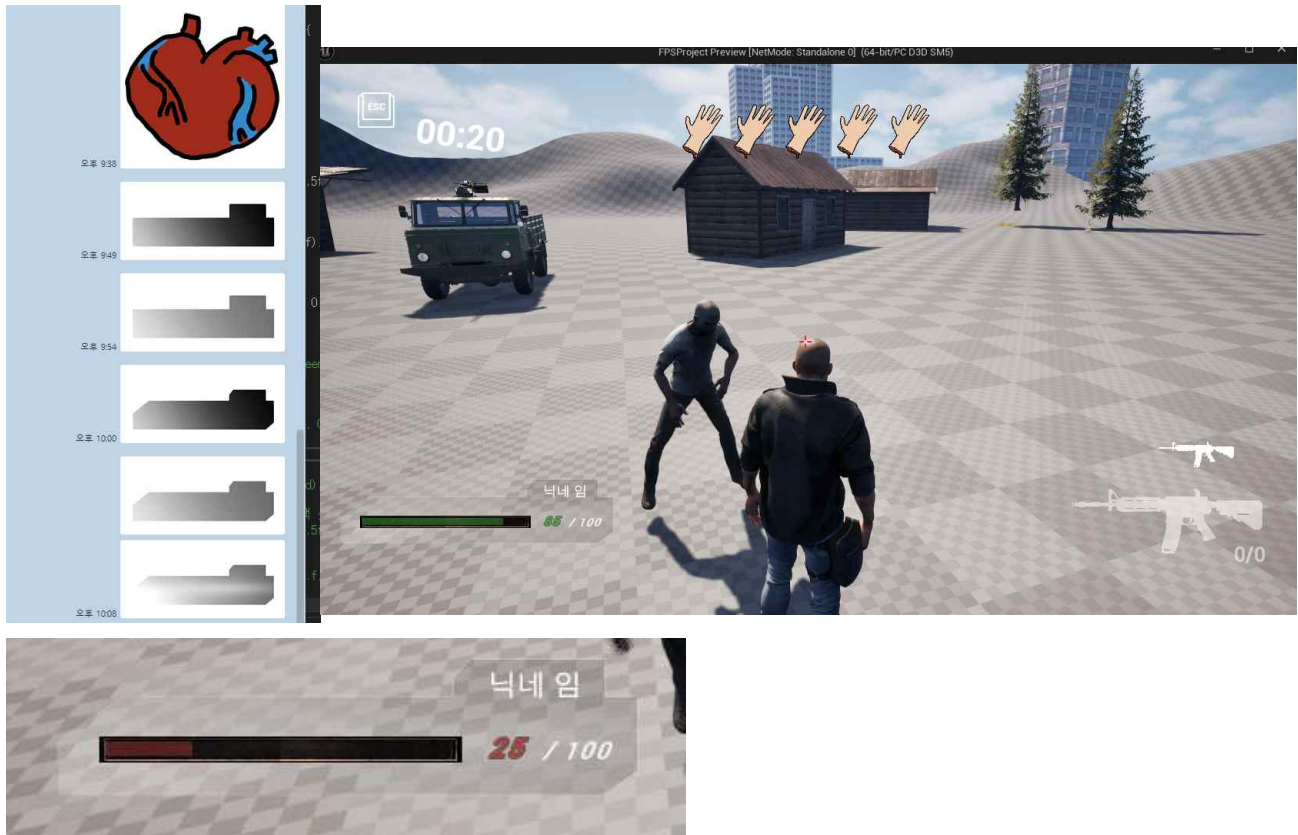
15주 차 보고서

(2026년 4 월 5 일 ~ 2026년 4 월 11 일)

작성자

안윤진

이번 주 한 일



체력바 패널로 쓰려고 다른 ui 참고해서 좀 간단하게 제작해봤다.

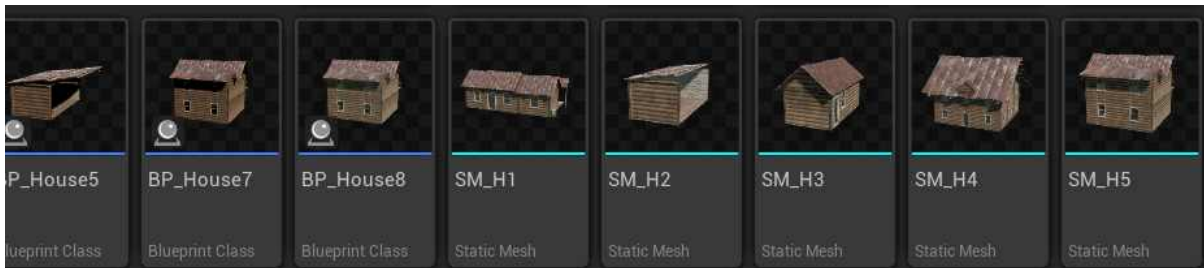
체력바를 플레이어의 실제 체력 변화에 맞게 연결하였다.

[ABaseZombie::OnAttackMontageEnded](#)에서 플레이어에게 데미지를 주면 그 밑에서 체력세팅 함수를 부르게 설정하였다. 플레이어 폰의 함수 부름 -> 베이직 위젯 안의 체력 세팅 함수 실행

이때 체력에 따라 초록/주황/빨강으로 체력바와 글자 색이 변하게 세팅하였다.

-> 좀비의 공격 후 체력이 닳는 게 반영되는 과정에서 딜레이가 존재한다. 후에 해결 예정

이번 주 한 일



돌 에셋을 추가했다. 좀 다른 영상들을 보면서 좀 더 조정해보았다... 결과가 뭐가 뻥 있는건 아니지만, 시간을 넣긴 했는데... 열심히 했는데 결과가 잘 모르겠다..

그리고 집 같은 경우 기존에 만들어둔 블프가 너무 용량이 커 조정에 딜레이가 걸리는 느낌을 받아, 문을 떼고, 스태틱 메쉬로 전부 다시 변환해준 후 다시 블프 문을 달아줬다.



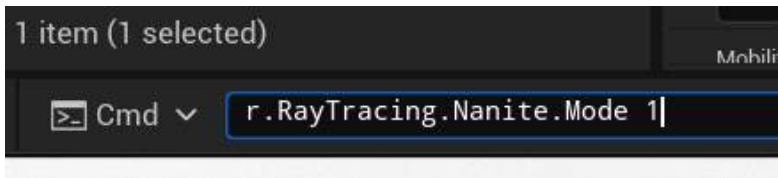
약간의 디테일도 넣어 줬다. 근데 아직 집 한참 남았다..

이번 주 한 일



원래 괜찮았다가 갑자기 에셋들에서 이런 거대한 검은 얼룩같은 그림자 발생. 사진보다 더 큰 얼룩들 많이 생김.

에셋 전체 삭제 및 다시 재다운로드 시도 -> 변화 없음.



인터넷에 찾아본 여러 방법 사용해도 해결되지 않았다.

에셋 전부 나나이트를 꺼버리니 그림자는 사라졌다. 그치만 제대로 된 해결법은 아닌 것 같다. 그 외에도 신우형이 알려준 방법 등 한 5가지 해보았는데 제대로 해결되지 않았다.

변위맵 쪽 문제 같은데, 자료를 찾아봐도 잘 모르겠다.

다음 주 할 일

기관총 사용 시 ui 제작

연기(매연효과) 나이가가라 이펙트 제작

특이사항